LocalHouseLLM Research

Independent AI Systems & Local Inference Research Division

Historical Evolution of GPU Inference Optimization

Report 03 · GPU Kernel & Systems History | Prepared by LocalHouseLLM Research | Confidential Technical Compendium

Historical Evolution of GPU Inference Optimization

LocalHouseLLM Research

Executive Summary

GPU-based inference began with naive usage of cuBLAS/cuDNN for transformer layers. Early solutions fused key operations (GEMM+transpose+softmax+GEMM) into custom kernels — notably NVIDIA's FasterTransformer (2019+) for BERT/GPT, which laid groundwork for high-performance inference. In 2022–2023, FlashAttention introduced a single-pass I/O-optimal attention kernel, cutting memory traffic by fusing operations and reaching near-peak throughput on Ampere GPUs. Subsequent refinements (FlashAttention-2, -3) exploited newer hardware (Hopper's Tensor Memory Accelerator and FP8) to double throughput. In parallel, inference runtimes adopted advanced scheduling: vLLM (2023) introduced continuous, iteration-level batching with PagedAttention to eliminate KV-cache waste, and Meta/SGLang (2024) added overlap schedulers to hide CPU work. Meanwhile, compiler stacks evolved: PyTorch's torch.compile (TorchInductor) emerged (2023–25) using Triton kernels and CUDA Graphs to fuse transformers, and frameworks like NVIDIA TensorRT-LLM (2023) and Meta AITemplate (2022) began generating fused CUDA/HIP code. By 2026, a rich ecosystem of kernel libraries, DSLs, and schedulers has formed to push LLM inference to hardware limits.

Complete Taxonomy of GPU Inference Optimizations

- **Kernel Fusion & Decomposition:** Merging multiple transformer ops (GEMM, softmax, bias, residual) into one kernel to eliminate memory round-trips; splitting large kernels (multi-head attention) into phases (prefill vs decode) for efficiency.
- **Thread/Block Strategies:** Warp specialization (dedicating warps for memory load vs compute), vectorized loads, register tiling, and occupancy tuning to fully utilize SMs.
- **Memory Hierarchy:** Using shared memory (tiling Q/K/V blocks), constant memory for weights, cache-local loads, and newer features like Hopper's cp.async/TMA to hide global-memory latency.
- **Precision and Compute Units:** Exploiting Tensor Cores (WMMA) in FP16 or FP8 to boost throughput, plus leveraging low-precision (int8/4) if hardware allows.

- **Launch/Execution Optimization:** Using CUDA Graphs to batch small launches into one replayable sequence, persistent kernels that loop internally, CUDA streams/async to overlap operations, and dynamic launch parameter tuning.
- **Scheduling & Batching:** Dynamic batching, continuous token-wise batching (vLLM-style), chunked prefill vs interleave decode steps, speculative multi-token decoding, and fairness-aware schedulers.
- **KV-Cache Management:** Block-sparse or paged memory layouts for key/value caches (e.g. vLLM's PagedAttention, FlashInfer's block-sparse format) to reduce fragmentation and enable sharing.
- **Memory Allocators:** Pre-allocating or pooling GPU memory (e.g. RAPIDS RMM), unified managed memory for oversubscription, pinned host memory for async DMA, and multi-GPU interconnect (NVLink) utilization.
- **Compiler/Auto-tuning:** DSLs and compilers (Triton, TorchInductor, AITemplate, XLA) that perform graph-level fusion, auto-schedule tiling, and generate optimized SASS/PTX. Autotuners search tile sizes and launch parameters for each hardware.
- **Hardware-specific Features:** Leveraging GPU architecture innovations (Hopper's TMA, warp-wide WGMMA, Ada/Blackwell features) and contrasting with other platforms (AMD ROCm's Matrix Cores, Apple MPS, Intel XPU).

CUDA Kernel Optimization Encyclopedia

Fused Attention Kernels: Merge the QK^T, softmax, and result-V steps into one kernel to keep intermediate data in fast memory. FlashAttention (2022) fuses these steps, yielding 3x–10x speedups over naive kernels. Uses carefully tiled loops and warp synchronizations to process a full attention block in one pass. Advantages: minimal global loads, high SM utilization. Drawbacks: complex code, hardware-sensitive.

Parallel Tiling (FA-2): Distributes even a single attention head across multiple thread-blocks. FlashAttention-2 (2023) boosts occupancy on long sequences. Each block loads different KV tile segments; warps cooperate on partial sums and softmax. Improves occupancy and throughput (up to 2x FlashAttention).

Warp Specialization & Pipelining: Assigns different warps within a block to distinct tasks. FlashAttention-3 (2024) on Hopper uses one warp group to do GEMM loads via TMA into shared memory, while another does the matmul softmax computation. Asynchronous overlap hides load latency (75% of peak achieved). Applicable only on architectures with async DMA (H100+).

Persistent Threads: Launch a grid of threads once per stream and loop over data chunks inside the kernel rather than relaunching, amortizing launch overhead for many small operations. Useful in ultra-low-latency or streaming scenarios.

Cooperative Groups & Collective Ops: CUDA cooperative groups or warp-level primitives accelerate reductions (softmax) and multi-GPU sync, reducing register pressure by sharing partial results intra-warp.

Register Tiling & Unrolling: Structure loops so each thread holds a tile of Q/K/V in registers; unroll reduction loops to reduce loop overhead and increase ILP. Uses more registers (possibly reducing occupancy) but boosts throughput.

Shared-Memory Blocking: Load blocks of K and V into shared memory to reuse across multiple threads computing Q*KV products, reducing redundant loads at the cost of shared memory usage. Must avoid bank conflicts via padding/transpose

layouts.

Half-Precision & WMMA: Use __half and WMMA API for block GEMM on Tensor Cores. Gains 8–16x arithmetic throughput but limited by tensor core shape.

Memory Coalescing & Vector Loads: Align data and use float4/half4 loads so adjacent threads access consecutive memory, improving effective bandwidth.

Minimize Divergence: Avoid branching inside warps; use predicated writes and masking instead of conditionals.

Occupancy Tuning: Choose blockDim and gridDim to maximize SM occupancy (~80%). Over-allocating registers may hurt by dropping occupancy.

Launch Configuration Tuning: Empirically search threadblock/tile sizes for peak throughput on each GPU; Triton's autotuner does this automatically.

PTX/SASS Tricks: Hand-vectorize or use PTX shuffle/mad instructions; inspect SASS to replace sequences with FMA or dp4a. Fragile but can squeeze out last few percent.

Multi-Instance GPU (MIG) Utilization: Running separate model instances on MIG slices avoids cross-request fragmentation, each slice with its own memory allocator. Tradeoff: inflexibility (large models need the full GPU).

Triton Kernel Optimization Encyclopedia

High-Level Tiling DSL: Triton lets developers write kernels in Python with logical tiles; the compiler maps these to threads/blocks per GPU, offering performance portability across NVIDIA, AMD, and Intel GPUs.

Autotuning: Triton can auto-search tile sizes, block shapes, and loop unroll factors. vLLM's Triton backend uses microbenchmarks to choose the best tile for each workload.

Kernel Fusion in Triton: Multiple operations can be fused into one Triton kernel; autotuned Triton fusion kernels rival custom CUDA in speed.

Portability Tricks: Triton lacks warp-level sync and constant-memory caches, so kernels must avoid those. Triton achieves ~90–95% of hand-tuned CUDA performance in many cases, while being easier to maintain.

Static vs Persistent Launch: Triton's model often uses static launch grids. vLLM experimented with "persistent" grid wrappers or CUDA Graph capture of variable loops.

Autotuning Limitations: The official Triton autotuner sometimes overfits specific GPU/hotspots; architecture-specific tuning could help further.

Boundary Handling: Triton kernels often handle ragged sequences by padding or masked loads.

Example — Triton PagedAttention: A Triton paged-attention kernel for vLLM matches optimized CUDA, achieving ~106% of a competing library's speed on both NVIDIA and AMD via heavy tuning.

Engine Integration: Triton kernels work with frameworks via TorchInductor or vLLM's backend. Torch.compile/TorchInductor uses Triton for many fused kernels, wrapped in CUDA Graphs. vLLM uses Triton kernels especially on AMD/Intel or FP32.

Runtime Scheduling Encyclopedia

Continuous (Iteration-Level) Batching: New requests are injected each token-step; once a sub-batch finishes, the next queued requests join. Keeps GPUs busy on latency-sensitive small-batch workloads. vLLM pioneered this in 2023. This greatly raises throughput when serving many simultaneous users.

Overlap Scheduler: SGLang and recent vLLM overlap CPU batch preparation with GPU execution via CUDA events/streams, yielding near-zero idle time via double-buffering.

Fairness & Priority (FairBatching): Systems ensure neither short prefill nor long decode requests starve each other by enforcing a ratio or interleaving to bound tail latency.

Chunked Prefill / Decoupled Decode: Long-context prefill is split into chunks so decoding can start earlier, overlapping I/O and compute between stages.

Dynamic Batching (Traditional): Servers accumulate requests up to a max batch size or timeout; simpler than continuous batching but can cause idle gaps.

Request Prioritization: Schedulers may prioritize shorter or earlier requests (head-of-line scheduling, deadlines, weighted fair queuing).

Speculative Decoding Scheduling: Manages two model invocations per step when multi-token speculation is used, balancing speculative work with final model latency.

Multi-GPU Pipeline: Inference pipelined across GPUs (tensor-parallel or pipeline-parallel); scheduling becomes allocating work stages to GPUs and streaming tokens through.

Concurrency Control: CUDA streams ensure independent batches overlap; CUDA Graph capture can record multi-kernel sequences, though fragmentation risk sometimes leads engines to disable graphs by default.

Evaluation Pipelines: Pinning CPU/GPU affinity or NUMA placement minimizes PCIe latency; container runtimes may use MIG partitions to isolate workloads.

Memory Optimization Encyclopedia

Paged KV Cache (vLLM's PagedAttention): Breaks KV cache into equal-sized pages stored anywhere in GPU memory instead of one contiguous tensor per sequence, eliminating fragmentation. Cuts wasted KV memory from ~60–80% to near 0, enabling 2–4x throughput gains for vLLM.

Radix-Tree KV Cache (SGLang): Stores all KV entries in a radix tree keyed by token prefix, implicitly sharing prefixes across sequences. Pros: automatic prefix sharing without manual copy; Cons: CPU-side complexity.

Block-Sparse KV Storage (FlashInfer): Divides KV into big blocks and only materializes blocks that are used, reducing memory for sparse/offloaded KV; leads to ~30% faster KV access.

Unified Managed Memory (NVLink C2C): On platforms like GH200, GPU+CPU share an address space; models and KV caches can spill into CPU DDR transparently, allowing e.g. a Llama-70B model to run on a 96GB GPU by paging to 480GB CPU memory.

Memory Pooling & Caching: Caching allocators reuse freed GPU memory to avoid fragmentation; configuring allocator settings (max_split_size_mb, expandable_segments) reduces fragmentation.

Multi-Instance GPU (MIG): Partitioning the GPU into isolated slices eliminates allocator contention entirely, good for multi-model serving.

High-Bandwidth Memory (HBM) Tuning: Align data to 128-byte boundaries; use TMA loads to prefetch large chunks into shared memory; overlap host<->device transfers with cudaMemcpyAsync.

NUMA/GPU Topology: Allocate GPU buffers on the CPU socket closest to the GPU; arrange one process per GPU on Grace nodes to use NVLink C2C.

Zero-Copy and Pinned Memory: Map host memory for GPU access (zero copy) for small data; page-lock input/output tensors to speed async transfers.

Cache Locality Tricks: Layout KV cache so warps access contiguous memory; interleave batched sequences to reduce stride.

Sequence Length Batching: Pad sequences to a common length within a batch to maximize reuse and vectorization; mask unused tokens rather than using multiple kernels.

Multi-GPU KV Sharing: In pipeline-parallel inference, heads from the same layer across GPUs share KV via NCCL broadcasts or NVLink P2P copies.

Batch-Contiguous Layouts: Some engines require “batch-major” tensor order for improved strided access when stepping tokens.

Compiler Optimization Encyclopedia

TorchInductor (Torch.compile): PyTorch’s compiler generates fused GPU kernels via FX/TorchDynamo, emitting Triton or C++ code, typically applying CUDA Graphs to fuse launches. Advantages: easy to use, supports dynamic control-flow. Drawbacks: unpredictable graph breaks; performance relies on Triton’s capabilities.

Torch-TensorRT (AOT): Converts a PyTorch/ONNX model to a TensorRT engine, fusing ops including attention where supported. TensorRT-LLM is a branch specialized for LLMs with handcrafted fused attention kernels and efficient KV handling. Advantages: very high performance on NVIDIA GPUs. Drawbacks: requires AOT build/export, limited to NVIDIA hardware.

ONNX Runtime + Execution Providers: ONNX Runtime can target GPUs via providers (TensorRT EP, OpenVINO EP, MIGraphX EP), partitioning the graph across backends.

AOTCompile Engines (IREE, TVM): Target GPUs but have been less used for transformer inference; TVM can auto-schedule but often lags hand-optimized kernels on large matrices.

OpenXLA (XLA for PyTorch): Google’s XLA aggressively fuses and vectorizes kernels across the graph; used mainly on Google Cloud TPUs.

Meta AITemplate: Compiles PyTorch models into C++ CUDA/HIP code with heavy fusion (horizontal and vertical), claiming up to 12x speedup over PyTorch eager on A100; uses FlashAttention for NV and AMD-specific fused kernels.

Custom Graph Rewrite Engines: TensorFlow XLA, OpenVINO, or proprietary ones (CoreML) exist but are outside mainstream GPU inference.

Auto-vectorization and MLIR: MLIR-based compilers (used internally by TorchInductor and AITemplate) apply pattern rewrites and polyhedral scheduling; still research territory.

Hidden Implementation Tricks

- **Padding and Mask Tricks:** Some kernels allow misaligned loops by padding at kernel boundaries; internal parameters (like page_size) are hand-tuned.
- **Inline PTX and Special Instructions:** Using inline PTX for WarpGroup-MMA (WGMMA) vs WMMA changes performance significantly on Hopper (35% vs 80% peak utilization).
- **Constant Memory for Biases:** Loading small biases/rotary embeddings from constant memory saves bandwidth; underused because it limits flexibility.
- **Prefetch in CP.async:** Hopper's cp.async can prefetch next tile into shared memory while computing; FlashAttention-3 repurposes a warp to issue these loads in advance.
- **Circular Buffers for Streaming:** For very long KV caches, a circular buffer keeps only the last N tokens, streaming old ones out.
- **Prefetch Chaining (Parallel GQA):** For GQA on decoding, using a prefill-style GEMM kernel for the query-key dot can be faster than a naive decode kernel, yielding 2–3x speedup.
- **Unrolling & Software Pipelining:** Manual loop unrolling or double-buffering K/V loads in registers appears mostly in handwritten kernels.
- **"No-op" Lazy Initialization:** Kernel graphs/caches created lazily to spread out startup cost; Triton kernels pre-warmed via a dummy pass.
- **Link-Time Optimization:** Inspecting generated PTX (e.g. via TensorRT) can reveal suboptimal code, replaced with explicit wmma/mma instructions.
- **Block Dim Selection Macros:** Some code switches block/thread dims based on compile flags or detected GPU compute capability.
- **Benchmarks-Only Tweaks:** Compiler flags like -use_fast_math or cache-as-texture settings can give several percent speed.
- **Texture/Read-Only Data:** Reading weight matrices via __ldg() (read-only cache) can be faster than normal loads.
- **Syncless Softmax:** Some kernels compute softmax reductions without explicit sync by relying on warp convergence; subtle and specialized.

Comparative Benchmarks

FlashAttention vs. FlashInfer vs. FlashMLA: On large contexts, FlashInfer's block-sparse attention yields modest speedups (13–17%) over standard FlashAttention. FlashMLA-ETAP (2025) reports a 2.78x gain over vanilla FlashMLA for 64K tokens, and ~5x vs FlashAttention-3, on H20 GPUs. For vanilla multi-head attention, FlashAttention-3 (with FP8) is currently the fastest exact-kernel, reaching ~1.2 PFLOPS on H100.

CUDA Graphs vs Traditional Launches: Auto-regressive models at token-granularity involve thousands of kernel launches; CUDA Graphs collapse these into one launch, cutting CPU overhead by tens of percent on moderate batch sizes. In vLLM, enabling CUDA Graphs gave ~10–20% throughput boost at small batch (though

severe memory fragmentation can lock in wasteful allocations, so some engines disable them by default).

Persistent vs Fused Kernels: Persistence eliminates all launch overhead; benefit is largest when each kernel call is very short. Anecdotal reports suggest persistent Triton kernels yield 10–15% extra throughput on small contexts, at higher complexity and debugging cost.

Triton vs Handwritten CUDA: Triton auto-fuses and tunes most patterns, often reaching ~95% of a hand-crafted CUDA kernel's speed. In some benchmarks, torch.compile (Triton backend) matched or slightly exceeded TensorRT-LLM performance on Llama models. CUDA still wins for novel warp-level/memory features not exposed to Triton.

TorchInductor vs TVM vs TensorRT: torch.compile (Inductor) running on GPU was as fast or faster than static TensorRT engines for tested LLMs in some benchmarks. TVM/Apache generally falls behind (1.5x–2x slower than ONNXRuntime/ONNX-TensorRT) since its generic auto-scheduler doesn't yet exploit attention-specific fusions. TensorRT-LLM generally yields top throughput at static shapes with large batches; TorchInductor wins for ease-of-use and near-top speed.

Scheduler Comparisons: Continuous batching (vLLM/SGLang) vs naive batching: vLLM reports up to 10x throughput improvement for bursty traffic. Overlap schedulers cut idle gaps further (e.g. SGLang saw ~50% reduction in end-to-end latency vs a non-overlapped queue).

Memory Strategy Benchmark: PagedAttention vs naive KV allocation shows dramatic capacity gains: vLLM could serve ~2x more parallel 4K-token requests. Enabling unified memory (NVMM/RMM) on a GH200 lets a 70B model (140GB) run on 96GB HBM, whereas without it the load OOMs.

Research Lineage

CUDA Kernels: Early kernel work (2017–2019) focused on GEMM and basic primitives; from 2019, fused transformer kernels emerged via FasterTransformer. In 2022, FlashAttention introduced a new tiling strategy eliminating $O(N^2)$ memory traffic. FlashAttention-2 (late 2022) parallelized inside each head; FlashAttention-3 (2024) introduced warp-level pipelining on Hopper. Separately, Triton's general-purpose kernels (2023) represent a branch reaching similar performance heights, extended across vendors by 2026.

Triton Evolution: Started (2022) as a research DSL; by 2024 became PyTorch-inductor's GPU backend and vLLM's optional attention engine, advancing from naive templates to fully optimized, auto-tuned kernels.

Scheduling: Initially, serving engines batched only per request. In 2023 vLLM introduced PagedAttention with a continuous-insertion scheduling model; SGLang (2024) refined the scheduler to eliminate CPU/GPU sync gaps. Early batched servers (TensorFlow Serving, TRTIS) gave way to these modern designs.

KV Caching: Lineage from contiguous per-request caches (inefficient) to advanced schemes: Kwon et al. (vLLM, 2023) introduced paged caches; SGLang (2024) added its radix-trie approach; FlashInfer (2025) took a block-sparse angle. Earlier "abandoned" notions of CPU-streamed caches are now superseded by unified memory/NVMM.

Compilers: Classic approach was static TF or ONNX→engine. PyTorch 2.0 (2023) brought torch.compile. TVM had attention schedules (2022) but has less traction. Meta's AITemplate (2022) spurred interest in codegen-based fusion. The current trend is JIT/AOT fusion via MLIR as the de facto standard; earlier MLIR projects (Glow, nGraph) are effectively superseded.

Abandoned/Legacy: Older attempts like RecTransformers (sparsity patterns), FFT-based attention, and naive multi-kernel decoding are now mostly historical. XLA on GPU for LLM has seen limited adoption.

Future Research Directions

Future work is pushing boundaries in several directions: hardware trends (new instructions on Blackwell and beyond, PIM instructions), compiler automation (better autotuners, ML-guided search, joint CPU/GPU compiling), scheduling theory (RL-driven, cost-aware schedulers), kernel innovations (hardware-software co-design, adaptive precision, structured sparsity caches), portability (Triton for Apple Metal or Vulkan backends), memory and interconnect (fine-grained KV compression, software-managed L2/HBM caching, intelligent offload), and multimodal fusion (generalized tensor-scheduling for heterogeneous dataflows). Overall, the frontier lies at the intersection of architecture-aware coding, compiler automation, and systems scheduling to eliminate remaining inefficiencies in GPU inference.

Bibliography

Comprehensive references for all topics above include research papers and engineering blogs: vLLM and FlashInfer system papers; FlashAttention series papers; NVIDIA and engineering blogs; Triton/torch.compile documentation; and AITemplate/compilers literature.

LocalHouseLLM Research — Internal Technical Compendium